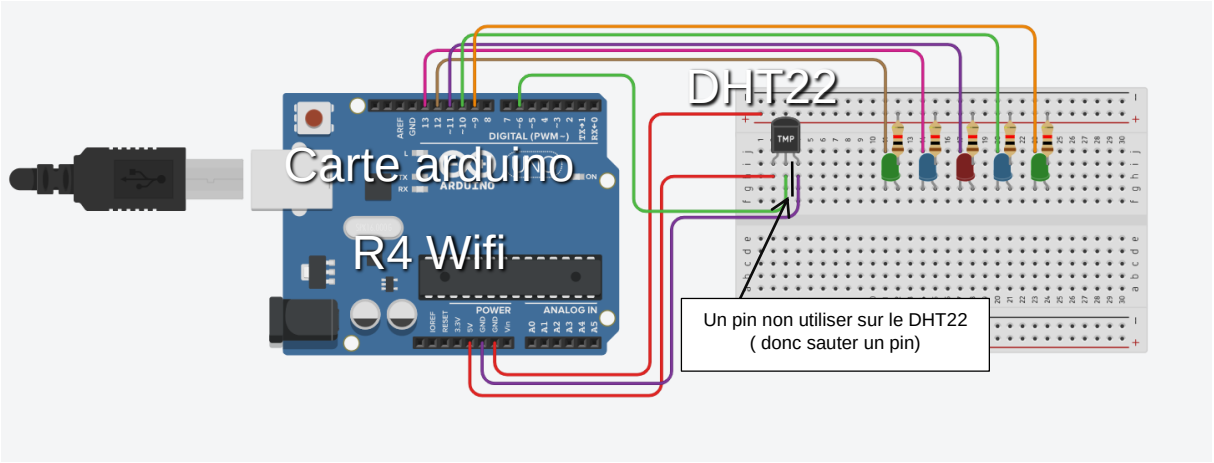
Contrôle d'une LED via une page Web avec Arduino Uno R4 WiFi

Concevoir un système de contrôle à distance sécurisé pour piloter des indicateurs lumineux (LEDs) et surveiller l'environnement (capteur DHT22) via une interface web.

Matériel utilisé :

- Arduino Uno R4 WiFi
- 5 LED
- 5 résistances de 220 ohms
- 1 capteur température DRT22
- Fils de connexion (9 câbles)
- Câble USB Type-C

Câblage :



Procédure :

1. Ouvrir l'Arduino IDE.
2. Installer la bibliothèque **WiFiS3** et **DHT sensor library par Adafruit**
3. Connecter l'Arduino R4 Wifi à l'ordinateur.
4. Copier le code du programme. (telechargement automatique) ou manuel voir page suivante **https://github.com/corentin-edu/arduino-r4-wifi/**
5. Modifier le nom et le mot de passe du réseau WiFi. (en haut du code partie « RÉSEAU & SÉCURITÉ »

```
char ssid[] = "name-wifi-2.4";
char pass[] = "password-wifi";
WiFiServer server(80);
```

6. Télé verser le programme sur l'Arduino.
7. Ouvrir le moniteur série pour obtenir l'adresse IP.
8. Entrer l'adresse IP dans un navigateur Web.

9. Autorisé la connexion via le monitor série grâce au « y » pour accepter ! (1 connexion à la fois autorisé)

10. Profitez du Dashboard web !

Fichier code : (telechargement automatique)

https://github.com/corentin-edu/arduino-r4-wifi/

Code pdf :

```
/* ************************************************************************
*/
/*   ARDUINO LED CONNECT V15.0 - BASED ON
V14.6                                */
/*   PATCHES: vitesse slider / connexion localStorage / DHT22 vi-
suel          */
/*   CIBLE : ARDUINO UNO R4
WIFI                                                 */
/* ************************************************************************
*/

#include <WiFiS3.h>
#include <DHT.h>

/*
==============================================================================
   DHT22 CONFIG
   ==============================================================================
= */
#define DHTPIN 6
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

/*
==============================================================================
   RÉSEAU & SÉCURITÉ
   ==============================================================================
= */
char ssid[] = "LR-Drone-1";
char pass[] = "225874120022587412";
WiFiServer server(80);

/*
==============================================================================
   HARDWARE
   ==============================================================================
= */
const int ledPins[5] = {12, 13, 11, 10, 9};
bool ledStates[5] = {false, false, false, false, false};
```

```
/*
==============================================================================
   SYSTÈME
   ==============================================================================
= */
int currentMode = 0;
int animSpeed = 250;
unsigned long prevTime = 0;
int animStep = 0;
unsigned long bootTime = 0;

float tempC = 0.0;
float humidite = 0.0;
unsigned long lastDHTRead = 0;
const unsigned long DHT_INTERVAL = 2000;

String activeSessionID = "";
String activeClientIP = "";
String pendingSessionID = "";
String pendingClientIP = "";
unsigned long pendingSessionTime = 0;
const unsigned long PENDING_TIMEOUT = 30000;
bool sessionDenied = false;
bool sessionKicked = false;

/*
==============================================================================
   PROTOTYPES
   ==============================================================================
= */
void initSerialMonitor();
void connectWiFi();
void animationEngine();
String generateSessionID();
void handleSerialCommands();
bool readHttpRequest(WiFiClient &client, String &req);
void serveInterface(WiFiClient& client);

/*
==============================================================================
   UTILITAIRES
   ==============================================================================
= */

void initSerialMonitor() {
  Serial.begin(9600);
  while (!Serial);
  Serial.println("\n\r*****************************************************");
```

```
  Serial.println("*    ARDUINO LED CONNECT V15.0                    *");
  Serial.println("*    Patches: vitesse / connexion / DHT22         *");
  Serial.println("*****************************************************");
  Serial.println("Commandes: accept/y, deny/n, kick, status");
}

void connectWiFi() {
  Serial.print("[📡] WiFi... ");
  WiFi.begin(ssid, pass);
  int counter = 0;
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
    if (counter++ > 40) {
      Serial.println("\n[❌] WiFi KO. Reboot...");
      NVIC_SystemReset();
    }
  }
  Serial.println(" OK !");
  Serial.print("📍 IP: ");
  Serial.println(WiFi.localIP());
  Serial.print("🔗 http://");
  Serial.println(WiFi.localIP());
  bootTime = millis();
}

/*
==============================================================================
   ANIMATIONS LEDs
   ==============================================================================
= */

void animationEngine() {
  if (currentMode == 0) return;
  unsigned long now = millis();
  if (now - prevTime >= (unsigned long)animSpeed) {
    prevTime = now;
    for (int i = 0; i < 5; i++) {
      digitalWrite(ledPins[i], LOW);
      ledStates[i] = false;
    }
    if (currentMode == 1) {
      const int seq[] = {12, 13, 11, 10, 9, 10, 11, 13};
      int pin = seq[animStep];
      digitalWrite(pin, HIGH);
      for (int i = 0; i < 5; i++) ledStates[i] = (ledPins[i] == pin);
      animStep = (animStep + 1) % 8;
    }
    else if (currentMode == 2) {
```

```
      if (animStep == 0) {
        digitalWrite(11, HIGH); ledStates[2] = true;
      } else if (animStep == 1) {
        digitalWrite(13, HIGH); digitalWrite(10, HIGH);
        ledStates[1] = true; ledStates[3] = true;
      } else if (animStep == 2) {
        digitalWrite(12, HIGH); digitalWrite(9, HIGH);
        ledStates[0] = true; ledStates[4] = true;
      } else if (animStep == 3) {
        digitalWrite(13, HIGH); digitalWrite(10, HIGH);
        ledStates[1] = true; ledStates[3] = true;
      }
      animStep = (animStep + 1) % 4;
    }
    else if (currentMode == 3) {
      int idx = animStep % 5;
      digitalWrite(ledPins[idx], HIGH);
      ledStates[idx] = true;
      animStep = (animStep + 1) % 5;
    }
    else if (currentMode == 4) {
      if (animStep == 0) {
        digitalWrite(12, HIGH); digitalWrite(11, HIGH); digitalWrite(9, HIGH);
        ledStates[0] = ledStates[2] = ledStates[4] = true;
      } else {
        digitalWrite(13, HIGH); digitalWrite(10, HIGH);
        ledStates[1] = ledStates[3] = true;
      }
      animStep = (animStep + 1) % 2;
    }
  }
}

/*
================================================================================
   SESSIONS & CONSOLE
   ================================================================================
= */

String generateSessionID() {
  return String(millis()) + String(random(1000, 9999));
}

void handleSerialCommands() {
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();
    cmd.toLowerCase();
    if (cmd == "accept" || cmd == "y") {
```

```
    if (pendingSessionID != "") {
      activeSessionID = pendingSessionID;
      activeClientIP = pendingClientIP;
      sessionDenied = false;
      Serial.print("\n[✅] Session OK ! ID: ");
      Serial.print(activeSessionID);
      Serial.print(" | IP: ");
      Serial.println(activeClientIP);
      pendingSessionID = "";
      pendingClientIP = "";
      pendingSessionTime = 0;
    } else {
      Serial.println("\n[❌] Aucune session en attente");
    }
  } else if (cmd == "deny" || cmd == "n") {
    if (pendingSessionID != "") {
      Serial.println("\n[⊘] Session refusée");
      sessionDenied = true;
      pendingSessionID = "";
      pendingClientIP = "";
      pendingSessionTime = 0;
    } else {
      Serial.println("\n[❌] Aucune session en attente");
    }
  } else if (cmd == "kick") {
    if (activeSessionID != "") {
      Serial.print("\n[🥾] Session kickée: ");
      Serial.print(activeSessionID);
      Serial.print(" | IP: ");
      Serial.println(activeClientIP);
      sessionKicked = true;
      activeSessionID = "";
      activeClientIP = "";
    } else {
      Serial.println("\n[❌] Aucune session active");
    }
  } else if (cmd == "status") {
    Serial.println("\n📊 ÉTAT DES SESSIONS");
    Serial.print("Active : ");
    Serial.print(activeSessionID);
    if (activeSessionID != "") {
      Serial.print(" | IP: ");
      Serial.println(activeClientIP);
    } else {
      Serial.println();
    }
    Serial.print("Pending: ");
    Serial.print(pendingSessionID);
    if (pendingSessionID != "") {
```

```
        Serial.print(" | IP: ");
        Serial.println(pendingClientIP);
      } else {
        Serial.println();
      }
    } else {
      Serial.println("\n[?] Commande inconnue");
      Serial.println("Commandes: accept/y, deny/n, kick, status");
    }
  }
  if (pendingSessionID != "" && (millis() - pendingSessionTime > PENDING_TI-
MEOUT)) {
    Serial.println("\n[⏱] Session en attente expirée (timeout 30s)");
    sessionDenied = true;
    pendingSessionID = "";
    pendingClientIP = "";
    pendingSessionTime = 0;
  }
}

/*
==============================================================================
   HTTP
   ==============================================================================
= */

bool readHttpRequest(WiFiClient &client, String &req) {
  client.setTimeout(100);
  req = client.readStringUntil('\r');
  return req.length() > 0;
}

/*
==============================================================================
   SETUP & LOOP
   ==============================================================================
= */

void setup() {
  initSerialMonitor();
  dht.begin();
  delay(2000);

  float h = dht.readHumidity();
  float t = dht.readTemperature();
  Serial.println("=== TEST DHT22 AU DEMARRAGE ===");
  if (isnan(h) || isnan(t)) {
    Serial.println("⚠ DHT22 non détecté");
    tempC = 0.0; humidite = 0.0;
```

```
  } else {
    Serial.print("Température : "); Serial.print(t); Serial.println(" °C");
    Serial.print("Humidité    : "); Serial.print(h); Serial.println(" %");
    tempC = t; humidite = h;
  }
  Serial.println("================================");
  Serial.println();

  for (int i = 0; i < 5; i++) {
    pinMode(ledPins[i], OUTPUT);
    digitalWrite(ledPins[i], LOW);
  }
  connectWiFi();
  server.begin();
}

void loop() {
  animationEngine();
  handleSerialCommands();

  if (millis() - lastDHTRead >= DHT_INTERVAL) {
    float h = dht.readHumidity();
    float t = dht.readTemperature();
    if (!isnan(h) && !isnan(t)) { tempC = t; humidite = h; }
    lastDHTRead = millis();
  }

  WiFiClient client = server.available();
  if (!client) return;

  String req;
  if (!readHttpRequest(client, req)) { client.stop(); return; }

  String sessionID = "";
  int sidPos = req.indexOf("sid=");
  if (sidPos != -1) {
    int endPos = req.indexOf(" ", sidPos);
    if (endPos == -1) endPos = req.length();
    sessionID = req.substring(sidPos + 4, endPos);
  }

  bool isRoot = (req.indexOf("GET / ") != -1);

  if (req.indexOf("GET /ping") != -1) {
    client.println("HTTP/1.1 204 No Content\r\nConnection: close\r\n\r\n");
    client.stop(); return;
  }

  if (req.indexOf("GET /request-session") != -1) {
```

```
    String clientIP = client.remoteIP().toString();
    if (activeSessionID != "" || pendingSessionID != "") {
      client.println("HTTP/1.1 200 OK\r\nContent-Type: applica-
tion/json\r\nConnection: close\r\n\r\n{\"status\":\"denied\"}");
    } else {
      pendingSessionID = generateSessionID();
      pendingClientIP = clientIP;
      pendingSessionTime = millis();
      Serial.print("\n🔔 NOUVELLE CONNEXION");
      Serial.print("\n   Session ID: "); Serial.println(pendingSessionID);
      Serial.print("   IP Client : "); Serial.println(clientIP);
      Serial.println("   Tapez 'accept' ou 'deny'");
      client.println("HTTP/1.1 200 OK\r\nContent-Type: applica-
tion/json\r\nConnection: close\r\n\r\n{\"status\":\"pending\",\"sid\":\"" +
pendingSessionID + "\",\"ip\":\"" + clientIP + "\"}");
    }
    client.stop(); return;
  }

  if (req.indexOf("GET /check-auth?sid=") != -1) {
    client.println("HTTP/1.1 200 OK\r\nContent-Type: application/json\r\nCon-
nection: close\r\n\r\n");
    if (sessionID == activeSessionID && activeSessionID != "") {
      client.print("{\"authorized\":true}");
    } else if (sessionDenied) {
      client.print("{\"authorized\":false,\"denied\":true}");
      sessionDenied = false;
    } else if (sessionKicked && sessionID != "") {
      client.print("{\"authorized\":false,\"kicked\":true}");
      sessionKicked = false;
    } else {
      client.print("{\"authorized\":false}");
    }
    client.stop(); return;
  }

  if (req.indexOf("GET /check-kicked?sid=") != -1) {
    client.println("HTTP/1.1 200 OK\r\nContent-Type: application/json\r\nCon-
nection: close\r\n\r\n");
    if (sessionKicked) {
      client.print("{\"kicked\":true}");
      sessionKicked = false;
    } else {
      client.print("{\"kicked\":false}");
    }
    client.stop(); return;
  }

  if (req.indexOf("/logout") != -1 && sessionID == activeSessionID) {
```

```
    activeSessionID = "";
    activeClientIP = "";
    client.println("HTTP/1.1 200 OK\r\nConnection: close\r\n\r\nOK");
    client.stop(); return;
  }

  if (!isRoot &&
      req.indexOf("GET /state") == -1 &&
      req.indexOf("GET /ping") == -1 &&
      req.indexOf("GET /request-session") == -1 &&
      req.indexOf("GET /check-auth") == -1 &&
      req.indexOf("GET /check-kicked") == -1 &&
      sessionID != activeSessionID) {
    client.println("HTTP/1.1 401 Unauthorized\r\nConnection: close\r\n\r\nIn-
valid Session");
    client.stop(); return;
  }

  // VITESSE — plage 100-400ms
  if (req.indexOf("speed=") != -1) {
    int sPos = req.indexOf("speed=") + 6;
    int end = req.indexOf(" ", sPos);
    if (end == -1) end = req.length();
    int v = req.substring(sPos, end).toInt();
    if (v >= 100 && v <= 400) animSpeed = v;
  }

  if (req.indexOf("/mode/0") != -1) {
    currentMode = 0;
    for (int i = 0; i < 5; i++) { digitalWrite(ledPins[i], LOW); ledStates[i]
= false; }
  } else if (req.indexOf("/mode/1") != -1) { currentMode = 1; animStep = 0;
prevTime = millis(); }
  else if (req.indexOf("/mode/2") != -1) { currentMode = 2; animStep = 0;
prevTime = millis(); }
  else if (req.indexOf("/mode/3") != -1) { currentMode = 3; animStep = 0;
prevTime = millis(); }
  else if (req.indexOf("/mode/4") != -1) { currentMode = 4; animStep = 0;
prevTime = millis(); }

  if (req.indexOf("/all/on") != -1) {
    currentMode = 0;
    for (int i = 0; i < 5; i++) { digitalWrite(ledPins[i], HIGH); ledStates[i]
= true; }
  }
  if (req.indexOf("/all/off") != -1) {
    currentMode = 0;
    for (int i = 0; i < 5; i++) { digitalWrite(ledPins[i], LOW); ledStates[i]
= false; }
```

```
  }

  for (int i = 0; i < 5; i++) {
    String pattern = "/toggle/" + String(i);
    if (req.indexOf(pattern) != -1) {
      currentMode = 0;
      ledStates[i] = !ledStates[i];
      digitalWrite(ledPins[i], ledStates[i] ? HIGH : LOW);
    }
  }

  if (req.indexOf("GET /state") != -1) {
    client.println("HTTP/1.1 200 OK\r\nContent-Type: application/json\r\nCon-
nection: close\r\n\r\n{\"l\":[");
    for (int i = 0; i < 5; i++) {
      client.print(ledStates[i] ? "1" : "0");
      if (i < 4) client.print(",");
    }
    client.print("],\"m\":"); client.print(currentMode);
    client.print(",\"s\":"); client.print(animSpeed);
    client.print(",\"u\":"); client.print((millis() - bootTime) / 1000);
    client.print(",\"t\":"); client.print(tempC, 1);
    client.print(",\"h\":"); client.print(humidite, 1);
    client.print("}");
  }
  else if (isRoot) { serveInterface(client); }
  else { client.println("HTTP/1.1 404 Not Found\r\nConnection:
close\r\n\r\n"); }

  delay(1);
  client.stop();
}

/*
==============================================================================
   PAGE WEB V15.0 — BASE V14.6 STRICTE + 3 PATCHES UNIQUEMENT
   ==============================================================================
= */

void serveInterface(WiFiClient& client) {
  client.println("HTTP/1.1 200 OK");
  client.println("Content-Type: text/html");
  client.println("Cache-Control: no-cache, no-store, must-revalidate");
  client.println("Connection: close");
  client.println();

  client.print(R"=====(
<!DOCTYPE html>
<html lang="fr">
```

```
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>🎮 Arduino LED Connect V15.0</title>
<style>
:root{--bg:#05070a;--panel:#0d1117;--acc:#58a6ff;--err:#f85149;--ok:#3fb950;--
warn:#d29922;--brd:#30363d;--txt:#c9d1d9}
*{box-sizing:border-box;margin:0;padding:0;font-family:'Consolas',monospace}
body{background:var(--bg);color:var(--txt);height:100vh;overflow:hidden;dis-
play:flex;flex-direction:column}
#auth-gate{position:fixed;inset:0;background:var(--bg);z-index:9999;dis-
play:flex;align-items:center;justify-content:center}
.auth-box{background:linear-gradient(135deg,#1a1f2e 0%,#0d1117 100%);pad-
ding:50px;border-radius:20px;border:2px solid var(--acc);box-shadow:0 10px
40px rgba(88,166,255,0.3);width:90vw;max-width:500px;text-align:center}
.auth-box.denied{border-color:var(--err);box-shadow:0 10px 40px
rgba(248,81,73,0.3)}
.auth-box.accepted{border-color:var(--ok);box-shadow:0 10px 40px
rgba(63,185,80,0.3)}
.auth-box.kicked{border-color:var(--warn);box-shadow:0 10px 40px
rgba(210,153,34,0.3)}
.spinner{border:4px solid rgba(255,255,255,0.1);border-top:4px solid var(--
acc);border-radius:50%;width:50px;height:50px;animation:spin 1s linear infi-
nite;margin:20px auto}
@keyframes spin{0%{transform:rotate(0deg)}100%{transform:rotate(360deg)}}
.btn-auth{width:100%;padding:18px;background:linear-gradient(135deg,var(--
acc),#1f6feb);color:#000;font-weight:bold;font-size:16px;border:none;cur-
sor:pointer;border-radius:10px;transition:all 0.3s;text-transform:upper-
case;letter-spacing:1px;margin-top:10px}
.btn-auth:hover{transform:translateY(-2px);box-shadow:0 5px 20px
rgba(88,166,255,0.6)}
.btn-auth.danger{background:linear-gradient(135deg,var(--err),#da3633);co-
lor:#fff}
.session-info{background:rgba(88,166,255,0.1);border:1px solid var(--acc);bor-
der-radius:10px;padding:15px;margin:20px 0;font-size:13px;text-align:left}
.session-info div{margin:8px 0;display:flex;justify-content:space-between}
.session-info .label{color:#8b949e}
.session-info .value{color:var(--acc);font-weight:bold;font-family:monospace}
#container{display:grid;grid-template-columns:1fr 2fr 1fr;grid-template-
rows:1fr;height:100vh;gap:1px;background:var(--brd);transition:filter 0.3s}
#container.offline{filter:blur(3px);pointer-events:none}
.column{background:var(--bg);padding:20px;display:flex;flex-direction:co-
lumn;overflow-y:auto}
.led-monitor{background:linear-gradient(180deg,#0a0c10 0%,#000 100%);pad-
ding:40px 30px;border-radius:15px;display:flex;justify-content:space-
around;margin-bottom:20px;border:2px solid var(--brd);box-shadow:inset 0 2px
10px rgba(0,0,0,0.8);position:relative}
```

```
.preview-badge{position:absolute;top:10px;right:10px;back-
ground:rgba(88,166,255,0.2);border:1px solid var(--acc);color:var(--acc);pad-
ding:4px 10px;border-radius:5px;font-size:10px;font-weight:bold;letter-spa-
cing:1px;animation:pulse-badge 2s infinite}
@keyframes pulse-badge{0%,100%{opacity:0.7}50%{opacity:1}}
.led-cell{text-align:center;flex:1}
.led-light{height:25px;width:25px;margin:0 auto 10px;border-radius:50%;back-
ground:#0a0a0a;border:3px solid #1a1a1a;transition:all 0.15s;box-shadow:inset
0 2px 5px rgba(0,0,0,0.8)}
.led-light.active.v{background:radial-gradient(circle,#3fb950,#2ea043);border-
color:#3fb950;box-shadow:0 0 30px #3fb950,0 0 60px #3fb950,inset 0 0 10px
#2ea043;animation:pulse-green 1.5s infinite}
.led-light.active.b{background:radial-gradient(circle,#58a6ff,#1f6feb);border-
color:#58a6ff;box-shadow:0 0 30px #58a6ff,0 0 60px #58a6ff,inset 0 0 10px
#1f6feb;animation:pulse-blue 1.5s infinite}
.led-light.active.r{background:radial-gradient(circle,#f85149,#da3633);border-
color:#f85149;box-shadow:0 0 30px #f85149,0 0 60px #f85149,inset 0 0 10px
#da3633;animation:pulse-red 1.5s infinite}
@keyframes pulse-green{0%,100%{box-shadow:0 0 30px #3fb950,0 0 60px
#3fb950,inset 0 0 10px #2ea043}50%{box-shadow:0 0 40px #3fb950,0 0 80px
#3fb950,inset 0 0 15px #2ea043}}
@keyframes pulse-blue{0%,100%{box-shadow:0 0 30px #58a6ff,0 0 60px #58a6ff,in-
set 0 0 10px #1f6feb}50%{box-shadow:0 0 40px #58a6ff,0 0 80px #58a6ff,inset 0
0 15px #1f6feb}}
@keyframes pulse-red{0%,100%{box-shadow:0 0 30px #f85149,0 0 60px #f85149,in-
set 0 0 10px #da3633}50%{box-shadow:0 0 40px #f85149,0 0 80px #f85149,inset 0
0 15px #da3633}}
.widget{background:var(--panel);border:1px solid var(--brd);border-ra-
dius:10px;padding:18px;margin-bottom:15px}
.widget h3{font-size:14px;margin-bottom:15px;color:var(--acc);border-bot-
tom:1px solid var(--brd);padding-bottom:8px;text-transform:uppercase;letter-
spacing:1px}
.btn{background:linear-gradient(135deg,#21262d,#161b22);border:2px solid var(-
-brd);color:#fff;padding:14px 18px;border-radius:8px;cursor:poin-
ter;width:100%;text-align:left;margin-bottom:10px;font-size:13px;font-
weight:600;display:flex;justify-content:space-between;transition:all 0.2s;po-
sition:relative;overflow:hidden}
.btn:hover{background:linear-gradient(135deg,#30363d,#21262d);border-co-
lor:var(--acc);transform:translateY(-2px);box-shadow:0 5px 15px
rgba(88,166,255,0.3)}
.btn.active{background:linear-gradient(135deg,var(--acc),#1f6feb);co-
lor:#000;border-color:var(--acc);box-shadow:0 0 20px rgba(88,166,255,0.5)}
.led-grid{display:flex;gap:8px;justify-content:space-between}
.led-btn{flex:1;background:linear-gradient(135deg,#1a1f2e,#0d1117);border:2px
solid var(--brd);color:#fff;padding:20px 8px;border-radius:10px;cursor:poin-
ter;font-size:11px;font-weight:600;text-align:center;transition:all 0.2s;posi-
tion:relative;overflow:hidden}
.led-btn:hover{transform:translateY(-3px);border-color:var(--acc);box-shadow:0
5px 20px rgba(88,166,255,0.3)}
```

```
.led-btn.active.v{border-color:var(--ok);box-shadow:0 0 15px var(--ok)}
.led-btn.active.b{border-color:var(--acc);box-shadow:0 0 15px var(--acc)}
.led-btn.active.r{border-color:var(--err);box-shadow:0 0 15px var(--err)}
.led-btn .key{position:absolute;bottom:5px;right:5px;font-size:9px;opa-
city:0.5;background:rgba(0,0,0,0.5);padding:2px 5px;border-radius:3px}
.log-area{background:#000;border:1px solid var(--brd);border-ra-
dius:5px;flex:1;font-size:11px;padding:10px;color:#8b949e;overflow-
y:scroll;line-height:1.6}
.stat-val{color:var(--ok);float:right;font-weight:bold}
#connection-status{position:fixed;top:10px;right:10px;padding:10px 20px;bor-
der-radius:20px;font-size:12px;font-weight:bold;z-index:1000;transition:all
0.3s;display:flex;align-items:center;gap:8px}
#connection-status.online{background:var(--ok);color:#000}
#connection-status.offline{background:var(--err);color:#fff}
input[type=range]{width:100%;height:8px;border-radius:5px;background:#333;ou-
tline:none;-webkit-appearance:none}
input[type=range]::-webkit-slider-thumb{-webkit-appearance:none;ap-
pearance:none;width:20px;height:20px;border-radius:50%;background:var(--
acc);cursor:pointer;box-shadow:0 0 10px rgba(88,166,255,0.5)}
/* PATCH DHT22 */
.dht-display{display:flex;gap:15px;margin-top:10px}
.dht-box{flex:1;background:linear-gradient(135deg,#1a2332,#0d1117);border:2px
solid var(--brd);border-radius:10px;padding:15px;text-align:center;transi-
tion:all 0.3s}
.dht-box:hover{border-color:var(--acc);box-shadow:0 0 15px
rgba(88,166,255,0.3)}
.dht-icon{font-size:24px;margin-bottom:4px}
.dht-value{font-size:26px;font-weight:bold;margin:5px 0;transition:color 0.5s}
.dht-label{font-size:10px;opacity:0.6;text-transform:uppercase;letter-spa-
cing:1px;margin-bottom:8px}
.dht-bar-bg{background:#1a1a2e;border-radius:5px;height:6px;width:100%;over-
flow:hidden;margin-top:8px}
.dht-bar{height:6px;border-radius:5px;transition:width 0.8s ease,background
0.5s}
#offline-screen{position:fixed;inset:0;background:linear-gra-
dient(135deg,rgba(5,7,10,0.98),rgba(248,81,73,0.1));backdrop-fil-
ter:blur(10px);z-index:9998;display:none;flex-direction:column;align-
items:center;justify-content:center;animation:fade-in 0.5s}
@keyframes fade-in{from{opacity:0}to{opacity:1}}
.offline-content{text-align:center;padding:40px;background:linear-gra-
dient(135deg,#1a1f2e,#0d1117);border:3px solid var(--err);border-ra-
dius:20px;box-shadow:0 0 60px rgba(248,81,73,0.5);animation:pulse-offline 2s
infinite}
@keyframes pulse-offline{0%,100%{transform:scale(1);box-shadow:0 0 60px
rgba(248,81,73,0.5)}50%{transform:scale(1.02);box-shadow:0 0 80px
rgba(248,81,73,0.7)}}
.offline-icon{font-size:100px;margin-bottom:20px;animation:shake 0.5s infi-
nite}
```

```
@keyframes shake{0%,100%{transform:translateX(0)}25%{transform:translateX(-
10px)}75%{transform:translateX(10px)}}
.offline-title{font-size:36px;font-weight:bold;color:var(--err);margin-bot-
tom:15px;letter-spacing:2px;text-transform:uppercase}
.offline-message{font-size:16px;color:#8b949e;margin-bottom:10px;line-
height:1.8}
.offline-dots{font-size:20px;color:var(--err);animation:blink 1.5s infinite}
@keyframes blink{0%,100%{opacity:1}50%{opacity:0.3}}
@media(max-width:768px){#container{grid-template-columns:1fr;grid-template-
rows:1fr auto 1fr}}
</style>
</head>
<body>
<div id="connection-status" class="online"><span>●</span><span id="status-
text">EN LIGNE</span></div>

<div id="offline-screen">
  <div class="offline-content">
    <div class="offline-icon">⚠</div>
    <div class="offline-title">CARTE ARDUINO HORS LIGNE</div>
    <div class="offline-message">Connexion perdue avec l'Arduino UNO R4
WiFi</div>
    <div class="offline-message" style="color:var(--warn)">⚡ Reboot possible
en cours...</div>
    <div class="offline-message">Détection automatique de reconnexion ac-
tive</div>
    <div class="offline-dots">● ● ●</div>
  </div>
</div>

<div id="auth-gate">
  <div class="auth-box" id="auth-content">
    <h2 style="color:white">🔒 ARDUINO LED CONNECT</h2>
    <p style="font-size:14px;margin:20px 0;opacity:0.8">Vérification de ses-
sion...</p>
    <div class="spinner"></div>
  </div>
</div>

<div id="container" style="display:none">
  <div class="column">
    <div class="widget">
      <h3>📊 ÉTAT</h3>
      <div style="font-size:12px">
        IP: <span id="ip-display" class="stat-val">---</span><br>
        UPTIME: <span id="uptime" class="stat-val">0s</span><br>
        PING: <span id="ping" class="stat-val">---</span><br>
        MODE: <span id="mode-name" class="stat-val">Manuel</span>
      </div>
```

```
    </div>
    <div class="widget">
      <h3>⚡ VITESSE</h3>
      <div style="font-size:11px;margin-bottom:10px">VITESSE: <span id="vval"
class="stat-val">Moyen</span></div>
      <input type="range" min="100" max="400" value="200" id="speedSlider"
oninput="debouncedSetSpeed(this.value)">
      <div style="display:flex;justify-content:space-between;font-
size:10px;opacity:0.5;margin-top:4px"><span>LENT</span><span>RA-
PIDE</span></div>
    </div>
    <div class="widget">
      <h3>🔄 MODES</h3>
      <button id="m1" class="btn" onclick="setM(1)">ALLER-RETOUR
<span>[1]</span></button>
      <button id="m2" class="btn" onclick="setM(2)">ESSUIE-GLACE
<span>[2]</span></button>
      <button id="m3" class="btn" onclick="setM(3)">CHENILLE
<span>[3]</span></button>
      <button id="m4" class="btn" onclick="setM(4)">ALTERNÉ
<span>[4]</span></button>
      <button class="btn" style="background:linear-gradient(135deg,var(--
err),#da3633)" onclick="setM(0)">STOP <span>[ESC]</span></button>
    </div>
    <div class="widget">
       <h3>🔐 SESSION</h3>
      <div style="font-size:11px;margin-bottom:10px;opacity:0.7">ID: <span
id="sid-display" style="color:var(--acc)">---</span></div>
      <button class="btn" style="background:linear-gradient(135deg,var(--
err),#da3633);text-align:center" onclick="logout()">DÉCONNEXION</button>
    </div>
  </div>

  <div class="column" style="border-left:1px solid var(--brd);border-right:1px
solid var(--brd)">
    <h2 style="margin-bottom:20px;letter-spacing:2px;text-align:center">⚙ MAIN
CONTROL PANEL</h2>
    <div class="led-monitor">
      <div class="preview-badge">● SIMULATION WEB</div>
      <div class="led-cell"><div id="L0" class="led-light v"></div><small>LED
1 (PIN 12)</small></div>
      <div class="led-cell"><div id="L1" class="led-light b"></div><small>LED
2 (PIN 13)</small></div>
      <div class="led-cell"><div id="L2" class="led-light r"></div><small>LED
3 (PIN 11)</small></div>
      <div class="led-cell"><div id="L3" class="led-light b"></div><small>LED
4 (PIN 10)</small></div>
      <div class="led-cell"><div id="L4" class="led-light v"></div><small>LED
5 (PIN 9)</small></div>
```

```
    </div>
    <div class="widget">
      <h3>💡 LEDS MANUELLES</h3>
      <div class="led-grid">
        <button id="b0" class="led-btn v" onclick="tg(0)"><div>LED 1</div><div
class="key">W</div></button>
        <button id="b1" class="led-btn b" onclick="tg(1)"><div>LED 2</div><div
class="key">X</div></button>
        <button id="b2" class="led-btn r" onclick="tg(2)"><div>LED 3</div><div
class="key">C</div></button>
        <button id="b3" class="led-btn b" onclick="tg(3)"><div>LED 4</div><div
class="key">V</div></button>
        <button id="b4" class="led-btn v" onclick="tg(4)"><div>LED 5</div><div
class="key">B</div></button>
      </div>
    </div>
    <div class="widget">
      <h3>⚡ GROUPÉ</h3>
      <div style="display:flex;gap:10px">
        <button class="btn" style="flex:1;background:linear-gra-
dient(135deg,var(--ok),#2ea043);color:#000;font-weight:bold;text-align:center"
onclick="allL(1)">TOUT ON [A]</button>
        <button class="btn" style="flex:1;background:linear-gra-
dient(135deg,var(--err),#da3633);color:#fff;font-weight:bold;text-align:cen-
ter" onclick="allL(0)">TOUT OFF [Z]</button>
      </div>
    </div>
    <div class="widget">
      <h3>🌡 CAPTEUR DHT22</h3>
      <div class="dht-display">
        <div class="dht-box">
          <div class="dht-icon" id="temp-icon">🌡</div>
          <div class="dht-label">Température</div>
          <div class="dht-value" id="temp-display">--°C</div>
          <div class="dht-bar-bg"><div class="dht-bar" id="temp-bar"
style="width:0%;background:var(--acc)"></div></div>
        </div>
        <div class="dht-box">
          <div class="dht-icon" id="hum-icon">💧</div>
          <div class="dht-label">Humidité</div>
          <div class="dht-value" id="hum-display">--%</div>
          <div class="dht-bar-bg"><div class="dht-bar" id="hum-bar"
style="width:0%;background:var(--acc)"></div></div>
        </div>
      </div>
    </div>
  </div>

  <!-- COLONNE LOGS — STYLE ORIGINAL V14.6 RESTAURÉ -->
```

```
  <div class="column">
    <h3>📝 LOGS</h3>
    <div id="logs" class="log-area">[SYSTEM] Initialisation...<br></div>
    <div style="margin-top:20px;font-size:10px;opacity:0.4;text-align:cen-
ter">ARDUINO LED CONNECT V15.0 © 2026</div>
  </div>
</div>

<script>
// ================================================================
// VARIABLES — identiques à V14.6 original
// ================================================================
let sid="",clientIP="",m=0,s=250,states=[0,0,0,0,0];
let isOnline=false,pingTimeout=null,speedTimer=null;
let offlineCount=0,reconnectAttempt=0,kickCheckInterval=null,offline-
Detected=false;
const modeNames=['Manuel','Aller-retour','Essuie-glace','Chenille','Alterné'];

// PATCH CONNEXION
const LS_KEY='arduino_sid',LS_IP='arduino_ip';

// PATCH VITESSE
function sliderToDelay(v){return 500-parseInt(v);}
function delayToSlider(d){return 500-d;}
function speedLabel(v){v=parseInt(v);if(v<150)return'Lent';if(v<280)re-
turn'Moyen';return'Rapide';}

// ================================================================
// SIMULATION LED LOCALE — indépendante, identique à V14.6
// ================================================================
let simStep=0,simInterval=null;

function ledColor(i){return i==2?'r':(i==1||i==3?'b':'v');}

function setSimLeds(activeIdxs){
  for(let i=0;i<5;i++){
    const led=document.getElementById('L'+i);
    if(!led) continue;
    led.classList.remove('active','v','b','r');
    if(activeIdxs.includes(i)) led.classList.add('active',ledColor(i));
  }
}

function simTick(){
  if(m==1){const seq=[0,1,2,3,4,3,2,1];setSi-
mLeds([seq[simStep%8]]);simStep++;}
  else if(m==2){const steps=[[2],[1,3],[0,4],[1,3]];setSi-
mLeds(steps[simStep%4]);simStep++;}
  else if(m==3){setSimLeds([simStep%5]);simStep++;}
```

```
  else if(m==4){if(simStep%2===0)setSimLeds([0,2,4]);else setSi-
mLeds([1,3]);simStep++;}
}

function startSim(mode,speed){
  stopSim();
  if(mode===0) return;
  simStep=0;
  simInterval=setInterval(simTick,speed);
}

function stopSim(){
  if(simInterval){clearInterval(simInterval);simInterval=null;}
}

// ================================================================
// LOGS — identique à V14.6
// ================================================================
function log(msg){
  const l=document.getElementById('logs');
  if(!l) return;
  const now=new
Date(),t=`${String(now.getHours()).padStart(2,'0')}:${String(now.getMi-
nutes()).padStart(2,'0')}:${String(now.getSeconds()).padStart(2,'0')}`;
  l.innerHTML+=`[${t}] ${msg}<br>`;
  l.scrollTop=l.scrollHeight;
  if(l.innerHTML.length>8000) l.innerHTML=l.innerHTML.slice(l.in-
nerHTML.length-8000);
}

// ================================================================
// PATCH DHT22 — couleurs dynamiques + barres de progression
// ================================================================
function updateDHT(t,h){
  const tD=document.getElementById('temp-display'),hD=document.getElement-
ById('hum-display');
  const tB=document.getElementById('temp-bar'),hB=document.getElement-
ById('hum-bar');
  const tI=document.getElementById('temp-icon'),hI=document.getElement-
ById('hum-icon');
  if(t==0){tD.innerText='--°C';tD.style.co-
lor='#8b949e';tB.style.width='0%';tI.innerText='🌡';}
  else{
    tD.innerText=t+'°C';
    tB.style.width=Math.min((t/50)*100,100)+'%';
    if(t<18){tD.style.color='#58a6ff';tB.style.background='#58a6ff';tI.inner-
Text='❄';}
    else if(t<=25){tD.style.color='#3fb950';tB.style.back-
ground='#3fb950';tI.innerText='☺';}
```

```
    else{tD.style.color='#f85149';tB.style.background='#f85149';tI.inner-
Text='⚠';}
  }
  if(h==0){hD.innerText='--%';hD.style.co-
lor='#8b949e';hB.style.width='0%';hI.innerText='💧';}
  else{
    hD.innerText=h+'%';
    hB.style.width=Math.min(h,100)+'%';
    if(h<30){hD.style.color='#d29922';hB.style.background='#d29922';hI.inner-
Text='🏜';}
    else if(h<=60){hD.style.color='#58a6ff';hB.style.back-
ground='#58a6ff';hI.innerText='💧';}
    else{hD.style.color='#a371f7';hB.style.background='#a371f7';hI.inner-
Text='🌊';}
  }
}

// ================================================================
// STATUS CONNEXION — identique à V14.6
// ================================================================
function updateStatus(online,ping=null){
  const status=document.getElementById('connection-status'),txt=document.getE-
lementById('status-text'),pingEl=document.getElementById('ping');
  const offlineScreen=document.getElementById('offline-screen'),container=do-
cument.getElementById('container');
  if(online){
    status.className='online';txt.innerHTML='EN LIGNE';
    if(pingEl){pingEl.innerHTML=ping+'ms';pingEl.className='stat-
val';if(ping>150)pingEl.className+=' warn';if(ping>300)pingEl.className+='
err';}
    offlineScreen.style.display='none';container.classList.remove('offline');
    if(!isOnline&&offlineDetected){log('🔄 Arduino reconnecté ! Recharge-
ment...');setTimeout(()=>location.reload(),1000);return;}
    if(!isOnline){log('✅ Connecté');isOnline=true;offlineCount=0;reconnec-
tAttempt=0;offlineDetected=false;}
    if(pingTimeout) clearTimeout(pingTimeout);
    pingTimeout=setTimeout(()=>updateStatus(false),5000);
  }else{
    status.className='offline';txt.innerHTML='HORS LIGNE';
    if(pingEl){pingEl.innerHTML='---';pingEl.className='stat-val err';}
    offlineScreen.style.display='flex';container.classList.add('offline');
    for(let i=0;i<5;i++){const led=document.getElement-
ById('L'+i);if(led)led.classList.remove('active','v','b','r');}
    if(isOnline){log('❌ Arduino déconnecté');isOnline=false;of-
flineCount=0;reconnectAttempt=0;offlineDetected=true;}
    offlineCount++;
    if(offlineCount>=6){reconnectAttempt++;if(reconnectAttempt>=4){log('🔄 Re-
chargement automatique...');setTimeout(()=>location.reload(),2000);}}
  }
```

```
}

// ================================================================
// PATCH CONNEXION — localStorage: reconnexion auto même navigateur
// ================================================================
async function init(){
  const savedSid=localStorage.getItem(LS_KEY);
  const savedIP=localStorage.getItem(LS_IP);
  if(savedSid){
    try{
      const resp=await fetch(`/check-auth?sid=${savedSid}`);
      const data=await resp.json();
      if(data.authorized){
        sid=savedSid;clientIP=savedIP||'Inconnu';
        log('🔑 Session restaurée');enterPanel();return;
      }else if(data.kicked){
        // Kick détecté au reload → on ignore et on refait une demande
        localStorage.removeItem(LS_KEY);localStorage.removeItem(LS_IP);
      }else if(data.denied){
        localStorage.removeItem(LS_KEY);localStorage.removeItem(LS_IP);
      }
    }catch(e){
      setTimeout(()=>init(),2000);return;
    }
  }
  requestSession();
}

async function requestSession(){
  try{
    const resp=await fetch('/request-session');
    const data=await resp.json();
    if(data.status==='pending'){
      sid=data.sid;clientIP=data.ip||'Inconnu';
      document.getElementById('auth-content').innerHTML=`
        <h2 style="color:white">⏳ AUTORISATION EN ATTENTE</h2>
        <p style="font-size:14px;margin:20px 0;opacity:0.8">En attente d'auto-
risation administrateur</p>
        <div class="session-info">
          <div><span class="label">Session ID:</span><span class="va-
lue">${sid}</span></div>
          <div><span class="label">Votre IP:</span><span class="va-
lue">${clientIP}</span></div>
          <div><span class="label">État:</span><span class="value" style="co-
lor:var(--warn)">EN ATTENTE</span></div>
        </div>
        <div class="spinner"></div>
        <p style="font-size:12px;opacity:0.6;margin-top:15px">L'administrateur
doit accepter votre connexion</p>
```

```
      `;
      waitForAuth();
    }else{
      document.getElementById('auth-content').innerHTML='<div style="font-
size:80px;margin:20px 0">⊘</div><h2 style="color:white">SESSION OCCU-
PÉE</h2><p style="font-size:14px;margin:20px 0">Un utilisateur est déjà con-
necté</p><button class="btn-auth" onclick="location.reload()">🔄 RÉES-
SAYER</button>';
      document.querySelector('.auth-box').classList.add('denied');
    }
  }catch(e){setTimeout(()=>location.reload(),3000);}
}

async function waitForAuth(){
  const check=setInterval(async()=>{
    try{
      const resp=await fetch(`/check-auth?sid=${sid}`);
      if(!resp.ok){clearInterval(check);location.reload();return;}
      const data=await resp.json();
      if(data.authorized){
        clearInterval(check);
        localStorage.setItem(LS_KEY,sid);localStorage.setItem(LS_IP,clientIP);
        document.getElementById('auth-content').innerHTML='<div style="font-
size:80px;margin:20px 0">✅</div><h2 style="color:white">ACCÈS AUTORISÉ</h2><p
style="font-size:14px;margin:20px 0">Chargement du panneau...</p>';
        document.querySelector('.auth-box').classList.add('accepted');
        setTimeout(()=>enterPanel(),1500);
      }else if(data.denied){
        clearInterval(check);
        document.getElementById('auth-content').innerHTML='<div style="font-
size:80px;margin:20px 0">❌</div><h2 style="color:white">ACCÈS REFUSÉ</h2><p
style="font-size:14px;margin:20px 0">L\'administrateur a refusé votre con-
nexion</p><button class="btn-auth danger" onclick="location.reload()">🔄 RE-
TENTER</button>';
        document.querySelector('.auth-box').classList.add('denied');
      }else if(data.kicked){clearInterval(check);showKickedScreen();}
    }catch(e){console.log('Erreur auth check',e);}
  },1000);
}

function enterPanel(){
  document.getElementById('auth-gate').style.display='none';
  document.getElementById('container').style.display='grid';
  document.getElementById('ip-display').innerText=clientIP;
  document.getElementById('sid-display').innerText=sid; // ligne à ajouter
  log('✅ Panneau de contrôle actif');
  log(`🔐 Session: ${sid}`);
  log(`📍 IP: ${clientIP}`);
  setInterval(sync,500);
```

```
  setInterval(checkPing,2000);
  kickCheckInterval=setInterval(checkKicked,2000);
}

async function checkKicked(){
  try{
    const resp=await fetch(`/check-kicked?sid=${sid}`);
    if(!resp.ok) return;
    const data=await resp.json();
    if(data.kicked){
      if(kickCheckInterval)clearInterval(kickCheckInterval);
      localStorage.removeItem(LS_KEY);localStorage.removeItem(LS_IP);
      stopSim();showKickedScreen();
    }
  }catch(e){}
}

function showKickedScreen(){
  stopSim();
  document.getElementById('container').style.display='none';
  document.getElementById('auth-gate').style.display='flex';
  document.getElementById('auth-content').innerHTML=`
    <div style="font-size:100px;margin:20px 0;animation:shake 0.5s infi-
nite">🚪</div>
    <h2 style="color:white">SESSION CLÔTURÉE</h2>
    <p style="font-size:16px;margin:20px 0;color:var(--warn)">Vous avez été
déconnecté par l'administrateur</p>
    <div class="session-info">
      <div><span class="label">Session ID:</span><span class="va-
lue">${sid}</span></div>
      <div><span class="label">Votre IP:</span><span class="value">${clien-
tIP}</span></div>
      <div><span class="label">État:</span><span class="value" style="co-
lor:var(--err)">KICK</span></div>
    </div>
    <button class="btn-auth" onclick="location.reload()">🔄 NOUVELLE CON-
NEXION</button>
  `;
  document.querySelector('.auth-box').className='auth-box kicked';
  log('🚪 Session kickée par admin');
}

async function checkPing(){
  const start=Date.now();
  try{
    const resp=await fetch('/ping',{cache:'no-cache'});
    if(resp.ok)updateStatus(true,Date.now()-start);
    else updateStatus(false);
  }catch(e){updateStatus(false);}
```

```
}

async function logout(){
  if(confirm('Déconnexion ?')){
    try{
      await fetch(`/logout?sid=${sid}`);
      localStorage.removeItem(LS_KEY);localStorage.removeItem(LS_IP);
      stopSim();log('👋 Déconnecté');
      if(kickCheckInterval)clearInterval(kickCheckInterval);
      location.reload();
    }catch(e){log('❌ Erreur déconnexion');}
  }
}

// ================================================================
// SYNC — mode/vitesse/DHT depuis Arduino, LEDs = simulation locale
// ================================================================
async function sync(){
  try{
    const resp=await fetch(`/state?sid=${sid}`,{cache:'no-cache'});
    if(!resp.ok){
      log('❌ Session invalide');
      localStorage.removeItem(LS_KEY);localStorage.removeItem(LS_IP);
      setTimeout(()=>location.reload(),1000);return;
    }
    const data=await resp.json();

    // Sync mode — relance sim si changement
    if(m!==data.m){
      m=data.m;
      document.getElementById('m1').classList.toggle('active',data.m==1);
      document.getElementById('m2').classList.toggle('active',data.m==2);
      document.getElementById('m3').classList.toggle('active',data.m==3);
      document.getElementById('m4').classList.toggle('active',data.m==4);
      document.getElementById('mode-name').innerText=modeNames[data.m];
      if(data.m===0){
        stopSim();
        // Mode manuel: sync LEDs depuis Arduino
        for(let i=0;i<5;i++){
          const led=document.getElementById('L'+i);
          const btn=document.getElementById('b'+i);
          if(!led||!btn) continue;
          led.classList.remove('active','v','b','r');
          btn.classList.remove('active','v','b','r');
          if(data.l[i]==1&&isOnline){led.classList.add('active',ledCo-
lor(i));btn.classList.add('active',ledColor(i));}
          states[i]=data.l[i];
        }
      }else{
```

```
        startSim(data.m,s);
      }
    }

    // Sync vitesse
    if(s!==data.s){
      s=data.s;
      const sliderVal=delayToSlider(s);
      document.getElementById('vval').innerText=speedLabel(sliderVal);
      document.getElementById('speedSlider').value=sliderVal;
      if(m>0) startSim(m,s);
    }

    document.getElementById('uptime').innerText=data.u+'s';
    updateDHT(data.t,data.h);

  }catch(e){console.log('Sync KO',e);updateStatus(false);}
}

// ================================================================
// CONTRÔLES
// ================================================================
function debouncedSetSpeed(v){
  if(!isOnline) return;
  const delay=sliderToDelay(v);
  s=delay;
  document.getElementById('vval').innerText=speedLabel(v);
  if(m>0) startSim(m,delay);
  if(speedTimer)clearTimeout(speedTimer);
  speedTimer=setTimeout(()=>fetch(`/?speed=${de-
lay}&sid=${sid}`).catch(()=>{}),200);
}

async function tg(i){
  if(!isOnline) return;
  m=0;stopSim();
  const newState=!states[i];
  states[i]=newState;
  const led=document.getElementById('L'+i);
  const btn=document.getElementById('b'+i);
  led.classList.remove('active','v','b','r');
  btn.classList.remove('active','v','b','r');
  if(newState){led.classList.add('active',ledColor(i));btn.classList.add('ac-
tive',ledColor(i));}
  document.getElementById('m1').classList.remove('active');
  document.getElementById('m2').classList.remove('active');
  document.getElementById('m3').classList.remove('active');
  document.getElementById('m4').classList.remove('active');
  document.getElementById('mode-name').innerText=modeNames[0];
```

```
  log(`🔄 LED ${i+1}: ${newState?'ON':'OFF'}`);
  fetch(`/toggle/${i}?sid=${sid}`).catch(()=>{});
}

async function setM(val){
  if(!isOnline) return;
  m=val;
  log(`🔄 Mode: ${modeNames[val]}`);
  document.querySelectorAll('.btn').forEach(b=>b.classList.remove('active'));
  if(val>0){document.getElementById('m'+val).classList.add('ac-
tive');startSim(val,s);}
  else stopSim();
  document.getElementById('mode-name').innerText=modeNames[val];
  fetch(`/mode/${val}?sid=${sid}`).catch(()=>{});
}

async function allL(v){
  if(!isOnline) return;
  m=0;stopSim();
  for(let i=0;i<5;i++){
    states[i]=v?1:0;
    const led=document.getElementById('L'+i);
    const btn=document.getElementById('b'+i);
    led.classList.remove('active','v','b','r');
    btn.classList.remove('active','v','b','r');
    if(v){led.classList.add('active',ledColor(i));btn.classList.add('ac-
tive',ledColor(i));}
  }
  document.getElementById('mode-name').innerText=modeNames[0];
  log(`⚡ Toutes LEDs: ${v?'ON':'OFF'}`);
  fetch(`/all/${v?'on':'off'}?sid=${sid}`).catch(()=>{});
}

document.addEventListener('keydown',e=>{
  if(e.repeat) return;
  if(document.getElementById('auth-gate').style.display!=='none'||!isOnline)
return;
  const k=e.key.toLowerCase();
  if(k==='w')tg(0);
  else if(k==='x')tg(1);
  else if(k==='c')tg(2);
  else if(k==='v')tg(3);
  else if(k==='b')tg(4);
  else if(k==='a')allL(1);
  else if(k==='z')allL(0);
  else if(k==='1'||k==='&')setM(1);
  else if(k==='2'||k==='é')setM(2);
  else if(k==='3'||k==='"')setM(3);
  else if(k==='4'||k==="'")setM(4);
```

```
  else if(k===='escape')setM(0);
  e.preventDefault();
});

window.addEventListener('load',init);
</script>
</body>
</html>
)=====");
}
```